



Lab Title: Insufficient Logging and Monitoring

Student Name: Nebeuwa Ifeanyichukwu Raphael

Student ID: 202520850LE

Course Name: BVWS103 – Owasp Top 10 Vulnerabilities
Exploitation Techniques

Instructor Name: AMMAR AMINU

Date of Submission: 15/02/2026

Executive Summary

In this lab, I learned how insufficient logging and monitoring can make a web application vulnerable to attacks. I created a simple PHP web application that logs user access and login attempts. I tested both successful and failed login attempts and checked how they were recorded in a log file.

I also added a monitoring feature that tracks multiple failed login attempts. When a user fails to log in three times within five minutes, the system locks the user out and logs an alert message. This helped me understand how logging and monitoring can detect suspicious activities like brute-force attacks.

Through this lab, I saw how proper logging and monitoring can help administrators detect unusual behavior early and respond before serious damage is done.

Lab Objective

The main objective of this lab was to understand the importance of logging and monitoring in web application security.

Specifically, I aimed to:

- Implement logging to record user access and login attempts.
- Monitor failed login attempts to detect suspicious behavior.
- Trigger an alert when multiple failed login attempts happen within a short time.
- Understand how logging and monitoring help prevent attacks such as brute-force login attempts.

Lab Environment

I performed this lab in a local development environment. I used:

- **Kali Linux** as my operating system.
- **XAMPP** to run Apache and host the PHP web application.
- **PHP** to write the application logic.
- A web browser to test login attempts.

All files (index.php, login.php, access.log, and failed_attempts.txt) were stored in the project folder inside the XAMPP htdocs directory. I accessed the application through <http://localhost/> on my browser.

Task 1: Exploit a Vulnerable Component

Step 1: Setting up the Vulnerable PHP Application

- Created a new folder ([Lab12_logging](#))

```
(raph@kali)-[/opt/lampp/htdocs]
└─$ sudo mkdir lab11_logging
[sudo] password for raph:

(raph@kali)-[/opt/lampp/htdocs]
└─$ ls
applications.html  img                lab11_logging
bitnami.css        index.php          lab17_Insecure_Deserialization
dashboard          lab10_VulnerableComponents  Lab2_HTMLInjection
favicon.ico        lab10_xss          Lab4_CommandInjection
```

```
(raph@kali)-[/opt/lampp/htdocs]
└─$ cd lab11_logging

(raph@kali)-[/opt/lampp/htdocs/lab11_logging]
└─$ sudo nano index.php

(raph@kali)-[/opt/lampp/htdocs/lab11_logging]
└─$ sudo nano logging.php

(raph@kali)-[/opt/lampp/htdocs/lab11_logging]
└─$ ls
index.php  logging.php

(raph@kali)-[/opt/lampp/htdocs/lab11_logging]
└─$
```

- Inside the folder, I created the following files:
 - Index.php (a page simulating a user interaction where actions are logged)

```
Session Actions Edit View Help
GNU nano 8.6 index.php *
<?php
session_start();
$log_file = __DIR__ . '/access.log'; // __DIR__ returns the current directory

// Check if the user is logged in (simulating a login system)
if (isset($_SESSION['user'])) {
    echo "<h1>Welcome, " . $_SESSION['user'] . "!</h1>";
} else {
    echo "<h1>Please log in</h1>";
    echo '<a href="login.php">Login</a>';
}

<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script> <!-- Updated jQuery version -->
// Log every access attempt
$log_message = "[" . date("Y-m-d H:i:s") . "] Access attempt from " . $_SERVER['REMOTE_ADDR'] . "\n";
file_put_contents($log_file, $log_message, FILE_APPEND);
?>
```

- Login.php (a page simulating user login)

```
raph@kali: /opt/lampp/htdocs/lab11_logging
Session Actions Edit View Help
GNU nano 8.6 logging.php *
}

// Log login attempts (both success and failure)
$log_message = "[" . date("Y-m-d H:i:s") . "] Login attempt by $input_user from " . $_SERVER['REMOTE_ADDR'] . "\n";
file_put_contents($log_file, $log_message, FILE_APPEND);
}

<form method="POST">
  <label for="username">Username:</label>
  <input type="text" id="username" name="username" required><br>
  <label for="password">Password:</label>
  <input type="password" id="password" name="password" required><br>
  <button type="submit">Login</button>
</form>
```

Step 2: Testing the logging Functionality

To test the login system, I accessed the login.php page through my browser using http://localhost/Lab12_Logging/login.php.

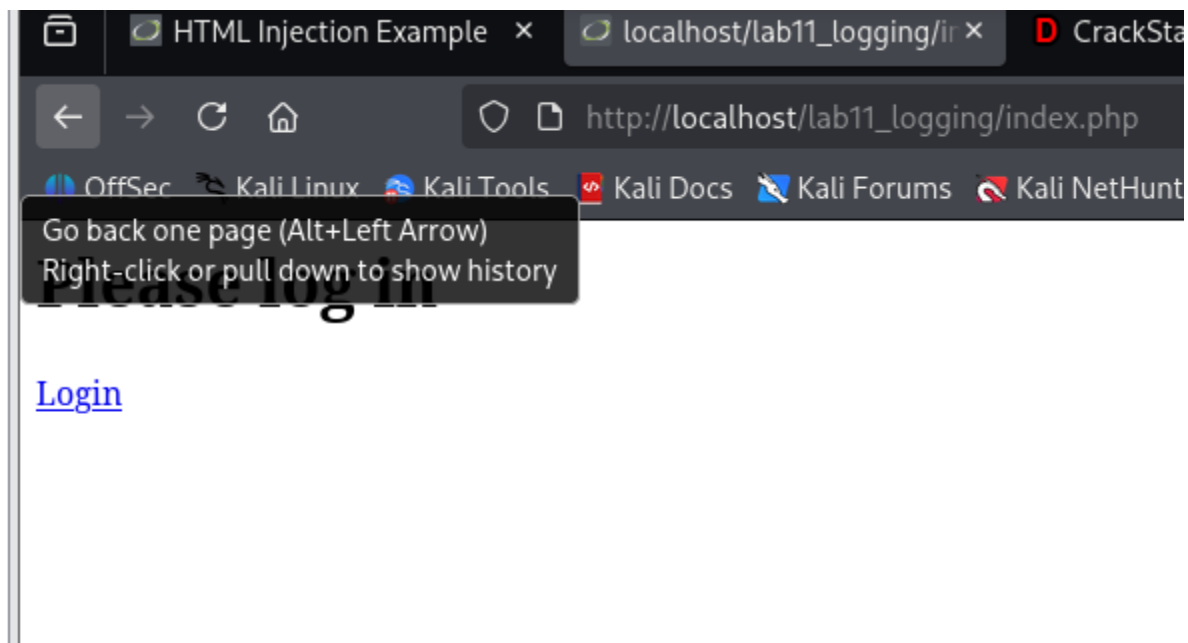
1. Testing with Correct Credentials

I entered: **Username:** admin **Password:** password123

After clicking the login button, the system displayed “**Login Successful!**”

When I checked the access.log file, I saw a log entry showing:

- The **date and time** of the login
- The **username (admin)**
- My **IP address** (for example, 127.0.0.1 since I was using localhost)



HTML Injection Example x localhost/lab11_logg

← → ↻ 🏠 http://localhost/lab11_

OffSec Kali Linux Kali Tools Kali Docs Kali Fo

Username:

Password:

Login

HTML Injection Example x localhost/lab11_logging/lc x

← → ↻ 🏠 http://localhost/lab11_logging/lc

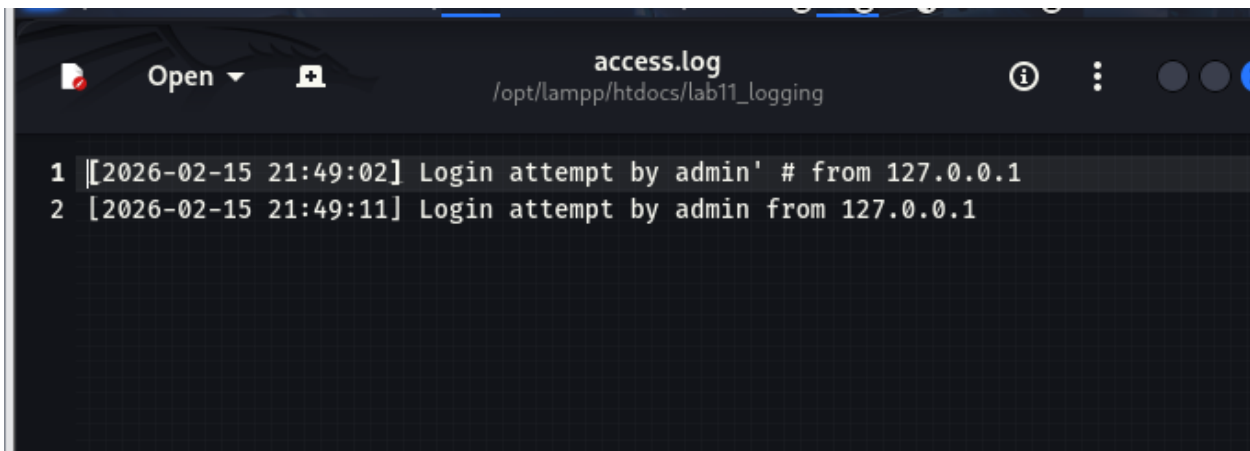
OffSec Kali Linux Kali Tools Kali Docs Kali Forums

Login Successful!

Username:

Password:

Login



2. Testing with Incorrect Credentials

Next, I entered wrong login details such as:

- **Username:** admin' #
- **Password:** wrongpass

After submitting, the system displayed **“Invalid Credentials!”**.

When I opened the access.log file again, I saw another entry recording the failed attempt. It still showed:

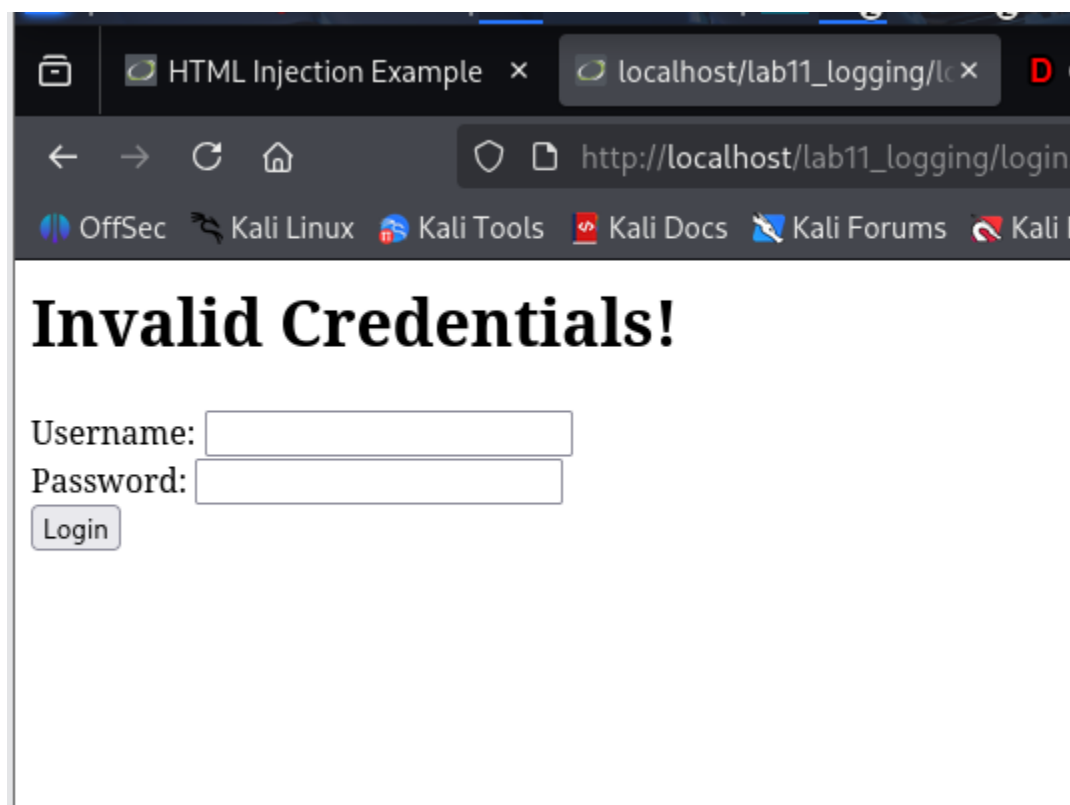
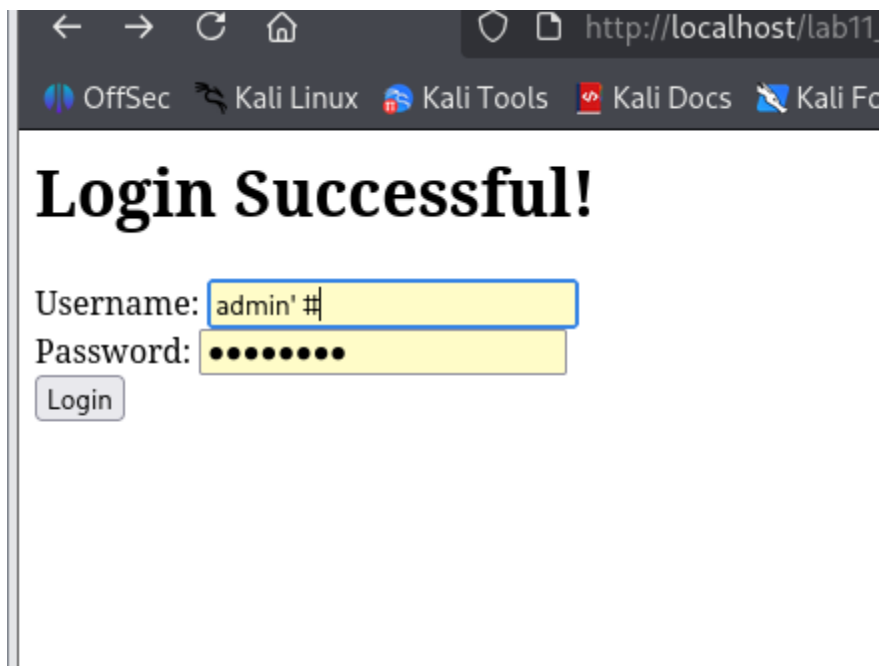
- The timestamp
- The username entered
- The IP address

What I Observed

After testing both successful and failed logins, the access.log file clearly recorded:

- The username used
- The exact date and time of the attempt
- The IP address of the user
- Whether the attempt was suspicious (if multiple failures occurred)

This shows that logging is working properly because every login attempt is being tracked and recorded.



A screenshot of a file editor window. The title bar shows 'access.log' and the path '/opt/lampp/htdocs/lab11_logging'. Below the title bar, there are tabs for 'checkfile_perm.sh', 'aisha', and 'access.log'. The 'access.log' tab is active, displaying a list of three log entries:

```
1 [[2026-02-15 21:49:02] Login attempt by admin' # from 127.0.0.1
2 [2026-02-15 21:49:11] Login attempt by admin from 127.0.0.1
3 [2026-02-15 21:53:07] Login attempt by admin' # from 127.0.0.1
```

Reflection Questions:

- **What information was recorded in the logs for each login attempt?**

For each login attempt, the log file recorded:

- ✓ The date and time of the attempt
- ✓ The username entered
- ✓ The IP address of the user (for example, 127.0.0.1 when using localhost)

This means every time someone tried to log in, the system saved important details that can later be reviewed.

- **How does logging help in detecting and monitoring suspicious activities?**

Logging helps by keeping a record of all user actions. If someone keeps trying to log in with wrong passwords many times, it can be clearly seen in the log file. This makes it easier to detect a possible brute-force attack.

With monitoring enabled, the system does not just record the activity, it also reacts to it. For example, after three failed login attempts within five minutes, the system logs an alert and locks the user out.

This helps the administrator:

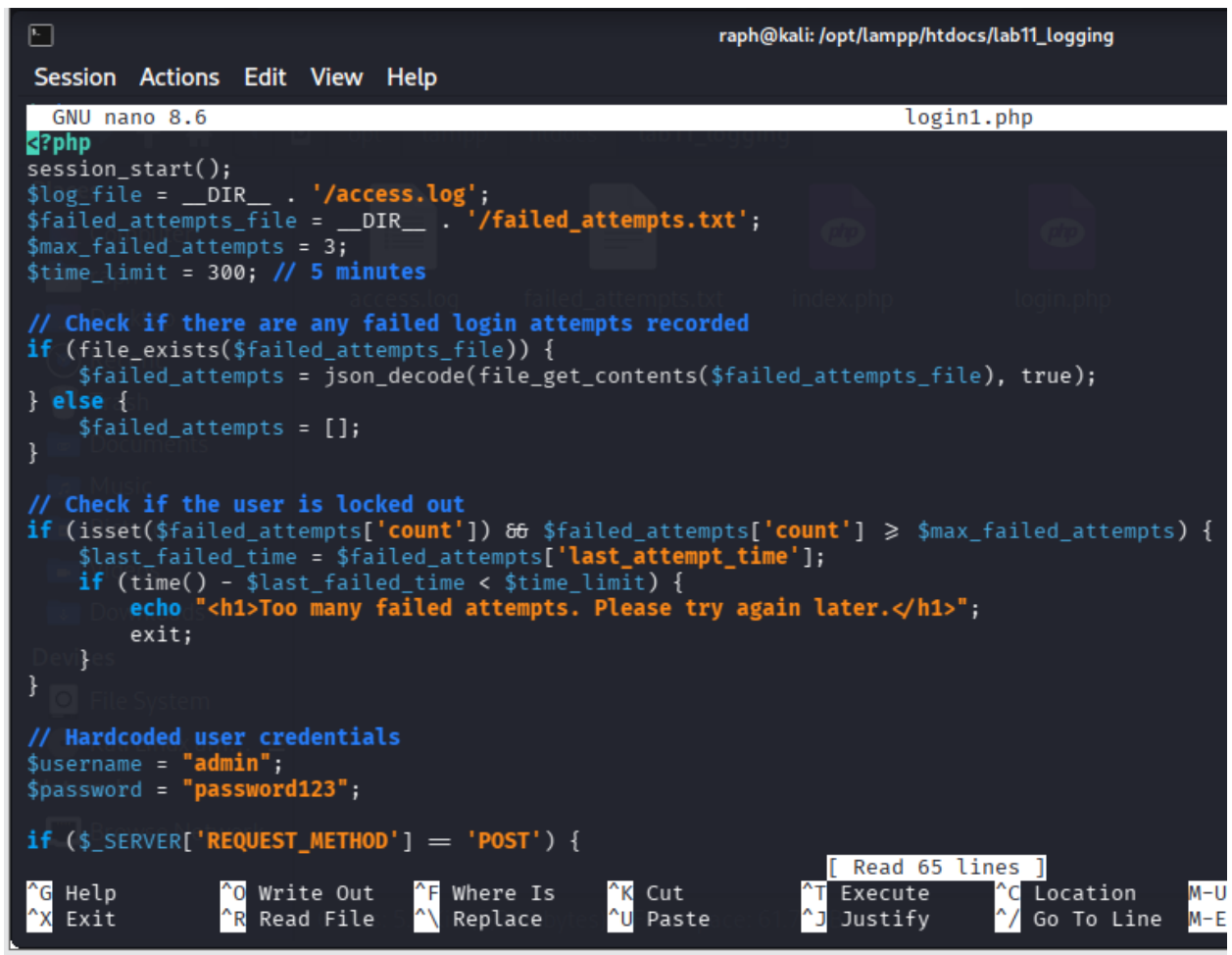
- ✓ Identify unusual behavior

- ✓ Know the IP address involved
- ✓ Take action before the attacker gains access

In simple terms, logging is like keeping security records, and monitoring is like watching those records in real time to stop suspicious activity early.

Task 2: Implemented Monitoring for Suspicious Activities

- **Modified the login.php page:** to monitor failed login attempts by counting them and logging an alert if there are multiple failed attempts within a short time period (e.g., 3 failed attempts within 5 minutes).



The screenshot shows a terminal window with the nano text editor open, editing the file `login1.php`. The user is `raph@kali` in the directory `/opt/lampp/htdocs/lab11_logging`. The code in the editor implements a security feature that monitors failed login attempts. It starts by initializing a session and defining log files: `access.log` and `failed_attempts.txt`. It sets a maximum of 3 failed attempts and a time limit of 300 seconds (5 minutes). The code then checks if the `failed_attempts.txt` file exists. If it does, it decodes the JSON data to get the current count of failed attempts. If the count is greater than or equal to the maximum allowed (3), and the time since the last failed attempt is less than the time limit, it echoes an error message: `<h1>Too many failed attempts. Please try again later.</h1>` and exits the script. Hardcoded user credentials for 'admin' with password 'password123' are also present. The code is triggered when the request method is POST.

```

GNU nano 8.6 login1.php
<?php
session_start();
$log_file = __DIR__ . '/access.log';
$failed_attempts_file = __DIR__ . '/failed_attempts.txt';
$max_failed_attempts = 3;
$time_limit = 300; // 5 minutes

// Check if there are any failed login attempts recorded
if (file_exists($failed_attempts_file)) {
    $failed_attempts = json_decode(file_get_contents($failed_attempts_file), true);
} else {
    $failed_attempts = [];
}

// Check if the user is locked out
if (isset($failed_attempts['count']) && $failed_attempts['count'] ≥ $max_failed_attempts) {
    $last_failed_time = $failed_attempts['last_attempt_time'];
    if (time() - $last_failed_time < $time_limit) {
        echo "<h1>Too many failed attempts. Please try again later.</h1>";
        exit;
    }
}

// Hardcoded user credentials
$username = "admin";
$password = "password123";

if ($_SERVER['REQUEST_METHOD'] = 'POST') {

```

At the bottom of the screen, there is a status bar with various keyboard shortcuts: `^G Help`, `^X Exit`, `^O Write Out`, `^R Read File`, `^F Where Is`, `^N Replace`, `^K Cut`, `^U Paste`, `^T Execute`, `^J Justify`, `^C Location`, `^_ Go To Line`, `M-U`, and `M-E`. A small box indicates "[Read 65 lines]".

Step 2: Test the Monitoring

To test the monitoring feature, I tried logging in with incorrect credentials three times in a row. For example:

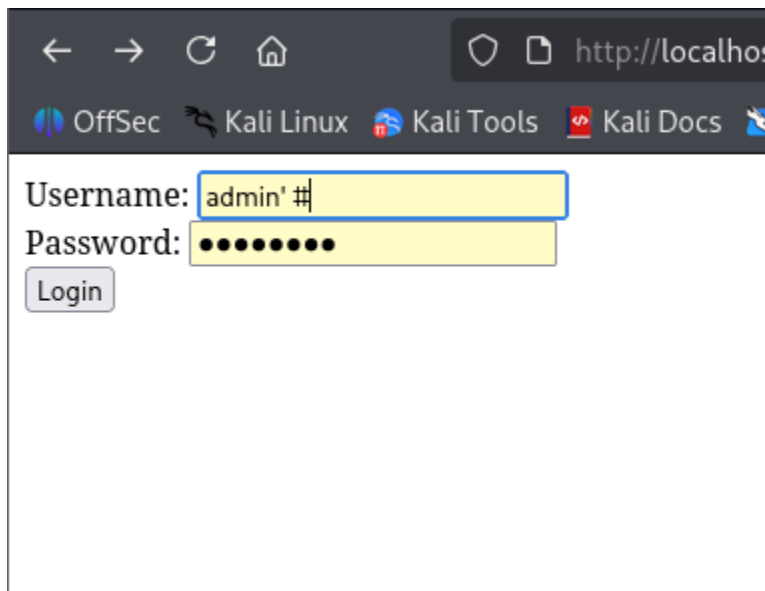
- **Username:** admin' #
- **Password:** wrong123

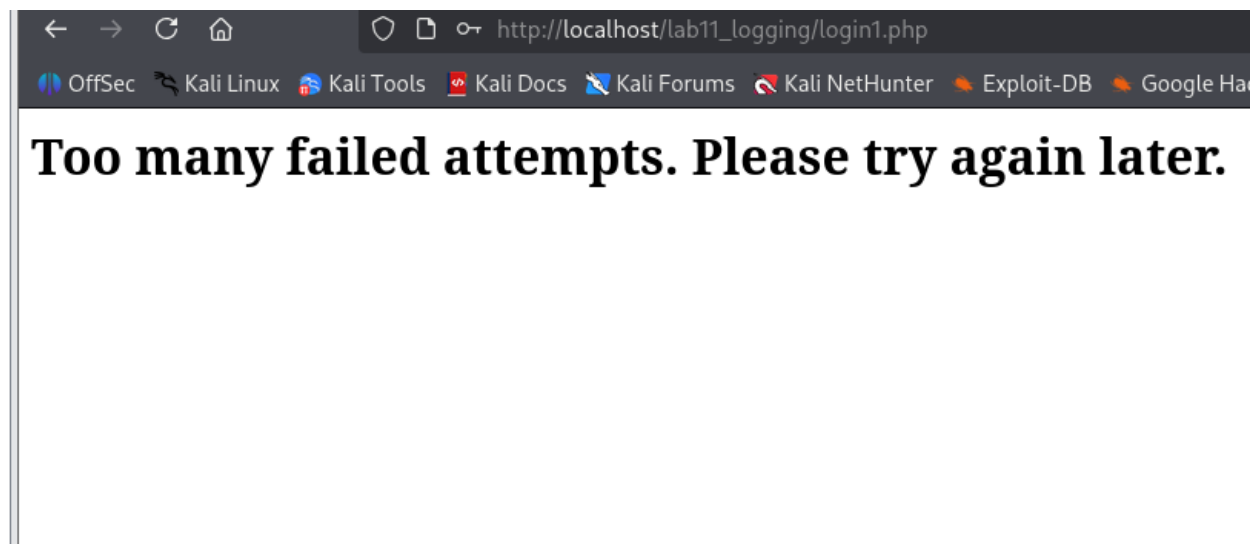
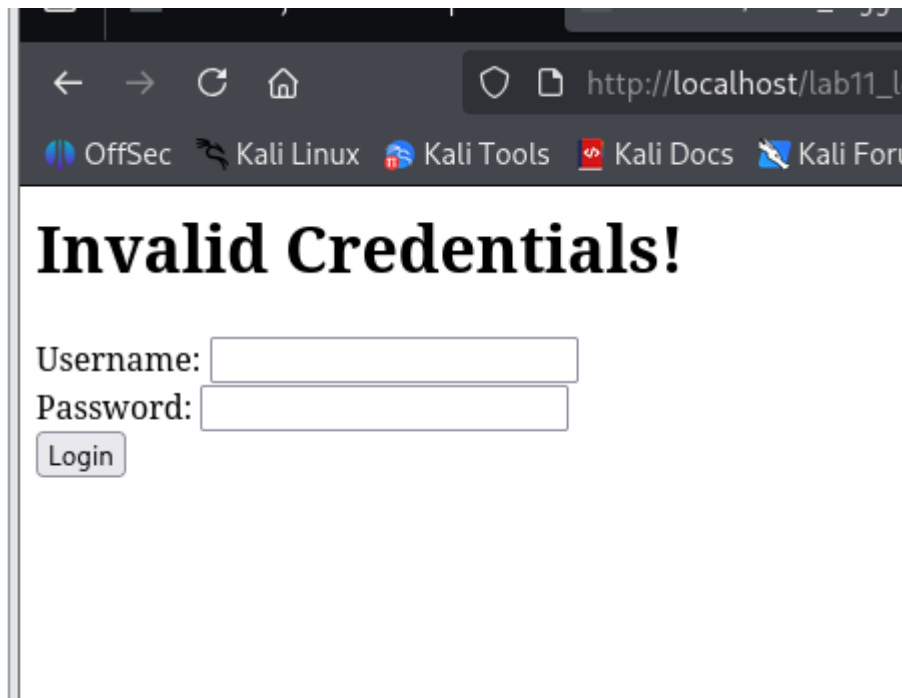
I repeated this process three times within a short period (less than five minutes).

After the third failed attempt, the system displayed the message:

“Too many failed attempts. Please try again later.”

This confirmed that the lockout mechanism was working properly. The system prevented further login attempts within the set time limit.





Reflection Questions:

- How did monitoring for failed login attempts improve the detection of suspicious activity?

Monitoring helped because the system counted failed login attempts and recognized when there were too many in a short time. Instead of just recording the attempts, it also triggered an alert. This made it easier to detect suspicious activity like a brute-force attack quickly.

- **Why is it important to have a mechanism for locking out users after multiple failed login attempts?**

Lockout is important because it prevents attackers from trying unlimited passwords. It protects user accounts and reduces the risk of someone guessing the correct password.

Index.php

```
<?php
session_start();

$log_file = __DIR__ . '/access.log'; // __DIR__ returns the current directory

// Check if the user is logged in (simulating a login system)
if (isset($_SESSION['user'])) {
    echo "<h1>Welcome, " . $_SESSION['user'] . " !</h1>";
} else {
    echo "<h1>Please log in</h1>";
    echo '<a href="login.php">Login</a>';
}

// Log every access attempt
$log_message = "[" . date("Y-m-d H:i:s") . "] Access attempt from " .
$_SERVER['REMOTE_ADDR'] . "\n";

file_put_contents($log_file, $log_message, FILE_APPEND);

?>
```

Login.php

```
<?php
session_start();

$log_file = __DIR__ . '/access.log'; // __DIR__ returns the current directory

// Hardcoded user credentials
$username = "admin";
$password = "password123";

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $input_user = $_POST['username'];
    $input_pass = $_POST['password'];

    if ($input_user === $username && $input_pass === $password) {
        $_SESSION['user'] = $input_user;
        echo "<h1>Login Successful!</h1>";
    } else {
        echo "<h1>Invalid Credentials!</h1>";
    }

    // Log login attempts (both success and failure)
    $log_message = "[" . date("Y-m-d H:i:s") . "] Login attempt by $input_user from " .
    $_SERVER['REMOTE_ADDR'] . "\n";
    file_put_contents($log_file, $log_message, FILE_APPEND);
}

?>
```

```
<form method="POST">

  <label for="username">Username:</label>

  <input type="text" id="username" name="username" required><br>

  <label for="password">Password:</label>

  <input type="password" id="password" name="password" required><br>

  <button type="submit">Login</button>

</form>
```

Access.log

```
[2026-02-15 21:49:02] Login attempt by admin' # from 127.0.0.1
[2026-02-15 21:49:11] Login attempt by admin from 127.0.0.1
[2026-02-15 21:53:07] Login attempt by admin' # from 127.0.0.1
[2026-02-15 21:56:09] Login attempt by admin from 127.0.0.1
[2026-02-15 21:56:33] Login attempt by admin' # from 127.0.0.1
[2026-02-15 22:15:06] Failed login attempt by admin' # from 127.0.0.1
[2026-02-15 22:21:39] Failed login attempt by admin' # from 127.0.0.1
[2026-02-15 22:21:43] Failed login attempt by admin' # from 127.0.0.1
[2026-02-15 22:21:43] ALERT: Suspicious activity detected. Too many failed login attempts
from 127.0.0.1
[2026-02-15 22:23:12] Login attempt by admin' # from 127.0.0.1
```

Failed_attempt.txt

```
{"count":3,"last_attempt_time":1771190503}
```

Key Takeaway

The key takeaway from this lab is that logging and monitoring are very important for web application security. Logging helps record user activities, while monitoring helps detect suspicious behavior like multiple failed login attempts. Without proper logging and monitoring, attacks can happen without being noticed.

Conclusion

In this lab, I learned how to implement logging and monitoring in a simple PHP web application. I was able to record login attempts and detect suspicious activity by tracking failed logins. This lab showed me that security is not only about preventing attacks but also about detecting and responding to them early. Proper logging and monitoring help protect the system and improve overall security.